

Banking Web Application Teaching Guide

Part 1: Architecture Overview, Backend Folder Structure, and Full Request Lifecycle

This guide is written as if you are preparing for a viva, interview, or project presentation. The goal is not only to know what the project does, but to explain why it is structured this way.

Project stack: React + Tailwind CSS + Axios + React Router on the frontend, and Spring Boot + Spring Security + JWT + JPA + MySQL on the backend.

1. Complete Project Architecture Overview

Purpose: The architecture is designed like a real banking system with clear separation of responsibilities. The frontend handles user interaction, the backend enforces business rules and security, and the database stores durable banking data such as users, accounts, beneficiaries, transactions, and fraud logs.

High-level architecture diagram:

```
User Browser
|
v
React Frontend
| (Axios HTTP requests with JWT token)
v
Spring Boot REST API
|
+--> Spring Security + JWT Filter
+--> Controllers
+--> Services
+--> Repositories (JPA)
v
MySQL Database
```

Why this structure is good for a capstone: It is simple enough to explain clearly, but strong enough to show enterprise thinking. Each layer has one job, which makes debugging, testing, and future expansion easier.

Main modules in the project:

Module	Purpose
Frontend	Shows pages, forms, dashboards, charts, and sends API requests.
Security Layer	Authenticates users and checks roles such as ADMIN and CUSTOMER.
Controller Layer	Receives HTTP requests and returns JSON responses.
Service Layer	Contains banking business logic like transfer validation and fraud scoring.
Repository Layer	Talks to MySQL through Spring Data JPA.
Database	Stores normalized banking records safely.

Interview explanation example: “I used a layered full-stack architecture so the UI, security, business logic, and persistence concerns remain separate. This makes the application more secure, easier to maintain, and closer to how enterprise banking products are built.”

If the examiner asks why not microservices, you can say: for a final-year capstone, a modular monolith is a smarter choice because it keeps deployment and explanation manageable while still following professional design principles.

Mentor note: always explain architecture from user flow first, then from code layers. That sounds much clearer in a viva.

2. Backend Folder Structure

Purpose: The backend is organized by responsibility. This helps a junior developer quickly know where to place code and helps interviewers see that the project follows clean architecture practices.

Backend package structure:

```
backend/src/main/java/com/banking/app/  
config/  
controller/  
dto/  
entity/  
exception/  
repository/  
security/  
service/  
BankingApplication.java
```

Folder-by-folder explanation:

Folder	What goes here	Why it matters
config	Beans, startup seed data, app-level configurations	Keeps shared setup separate from business logic.
controller	REST endpoints like /api/auth/login	Acts as the API entry point.
dto	Request and response classes	Prevents exposing entity classes directly.
entity	JPA database models	Represents tables and relationships.
exception	Custom errors and global handlers	Makes API error responses clean and consistent.
repository	Interfaces extending JpaRepository	Removes boilerplate SQL for standard queries.
security	JWT service, filter, user details service, security config	Centralizes authentication and authorization.
service	Core logic like transfer, profile update, fraud analysis	Where business rules live.

Why code is written this way: In banking software, mixing SQL, security, validation, and HTTP handling in one class becomes dangerous very quickly. By splitting the code into layers, we reduce risk and make every class easier to review.

Security angle: Sensitive logic such as token parsing, role validation, and password encoding is kept in dedicated security classes. This reduces accidental mistakes and makes audits easier.

Database interaction angle: Entities define the table structure, repositories access data, and services decide when to read or write. That means the controller does not directly manipulate the database.

Viva answer example: "I used DTOs so my API does not expose internal entity models directly. This gives me better control over validation, security, and response formatting."

Mentor note: if someone asks where transfer logic should live, the correct answer is the service layer, not the controller and not the repository.

3. Request Lifecycle: Frontend -> Backend -> Database -> Response

Purpose: This is the most important flow to understand for interviews. It shows how a user action travels through the whole system and returns safely with data.

Example scenario: customer logs in and opens the dashboard.

```
Step 1: User enters email and password in React login form
Step 2: Axios sends POST /api/auth/login to Spring Boot
Step 3: AuthController receives the request
Step 4: AuthService authenticates using Spring Security
Step 5: JWT token is generated and returned
Step 6: React stores the token
Step 7: React sends GET /api/customer/account with Authorization header
Step 8: JWT filter validates token before controller executes
Step 9: Controller calls service
Step 10: Service calls repository
Step 11: Repository reads data from MySQL
Step 12: DTO response is returned as JSON
Step 13: React renders dashboard cards and tables
```

Simple lifecycle diagram:

```
React Page
|
| Axios request
v
Controller
|
v
Service
|
v
Repository
|
v
MySQL
|
v
Repository -> Service -> Controller -> JSON Response -> React UI
```

How frontend-backend communication works:

The frontend uses Axios to send HTTP requests. For protected APIs, Axios attaches the JWT token in the Authorization header. The backend returns JSON, and React updates the page state using that response.

Example: when the dashboard loads, React calls `/api/customer/account`. The backend looks up the currently authenticated user from the JWT, finds the account in MySQL, maps it into an `AccountSummaryDto`, and returns it to the frontend.

How security works inside the lifecycle:

Security happens **before** business logic. The JWT filter checks whether the request has a valid token. If the token is invalid or missing, the request should never reach the protected controller logic.

```
Browser request with JWT
-> JwtAuthenticationFilter
-> extract username from token
-> load user details
-> validate token
-> put authentication into SecurityContext
-> allow request to continue
```

How database interaction works:

The service layer never writes raw SQL in controllers. Instead, it calls repository methods such as `findByEmail()`, `findByUser()`, or `save()`. Spring Data JPA converts those into SQL operations under the hood.

This is useful because the business logic stays readable. For example, transfer logic can focus on checking balance, updating account balance, storing the transaction, and writing a fraud log.

Why the response uses DTOs:

DTOs are safer than returning entities directly. They let us choose exactly which fields go to the client. For example, we never want to return hashed passwords, internal flags, or unnecessary relational data.

Interview explanation example:

“The request first enters the controller, then the service handles business rules, then the repository communicates with the database. After data is retrieved, the backend returns a DTO as JSON to the React frontend. This keeps the flow secure, testable, and maintainable.”

Mentor note: if you can clearly explain this flow, you will sound much stronger in a viva than someone who only describes screens.

Next Topics Preview

The next teaching pack should cover JWT flow visually, Spring Security configuration, annotations, module breakdown, transfer internals, fraud detection, and financial health score logic.

Use this line when presenting: “I can now walk you through how authentication, authorization, and secure money transfer are implemented step by step.”